

Risk-Aware Guarded Scheduling for Tail Latency Control in Distributed Systems

Robert Guy

Abstract

Modern distributed systems must make scheduling decisions under strict latency constraints while operating under dynamic workloads and shared-resource contention. Predictive (learning-based) scheduling can improve average-case performance by exploiting historical signals, yet small mispredictions can amplify into tail events that dominate end-to-end responsiveness [3]. In this work, we propose a *risk-aware guarded scheduling* framework that pairs a predictive optimization layer with an explicit runtime policy-enforcement layer: the scheduler derives risk signals from queue state and selectively overrides predictive decisions when the system enters a high-risk regime. This design is motivated by a broader systems trend toward centralized *visibility and control* and explicit *policy enforcement* to manage performance under uncertainty [2, 9]. We evaluate our method using a simulation framework modeling dynamic workloads and contention, comparing against predictive and lookahead baselines. Across five seeds, guarded scheduling reduces p99 latency by 47.3% relative to the predictive baseline while maintaining competitive mean latency.

1 Introduction

Latency-sensitive distributed systems must remain predictably responsive despite workload variability and shared-resource contention. In large services, rare tail events often dominate end-to-end latency and user experience, motivating designs that are explicitly *tail-tolerant* rather than optimized solely for the common case [3]. At the same time, modern deployments increasingly rely on shared, multi-tenant infrastructure, making *performance isolation* and enforceable control policies central to maintaining predictable behavior under contention [9]. Predictive scheduling has emerged as a promising direction for improving average-case performance by leveraging historical signals to anticipate near-term system behavior.

Learning-based resource management approaches demonstrate that models trained on prior system trajectories can guide online decisions effectively under typical load [10]. However, we observe that prediction alone is insufficient to reliably control tail latency. Under contention and distribution shifts, even small prediction errors can compound into queue imbalance and head-of-line blocking, leading to significant degradation in p95 and p99 latency.

Recent work in datacenter software stacks reinforces a complementary lesson: systems increasingly introduce explicit enforcement points to provide operators *visibility and control* and to apply policies at runtime. For example, mRPC relocates policy enforcement into a managed system service that can execute policy engines and arbitrate resource use without imposing unnecessary overheads from intermediary layers [2]. At the same time, measurement studies show that adding control functionality can introduce substantial and configuration-dependent overhead, emphasizing the need for lightweight mechanisms and careful placement of control logic [15].

These trends motivate a hybrid scheduling architecture that preserves the efficiency benefits of

prediction while providing a minimal enforcement mechanism to contain rare-but-high-impact states.

This paper explores such a hybrid approach. We propose a *risk-aware guarded scheduling* framework that monitors queue-derived risk signals and selectively overrides predictive decisions to contain emerging tail risk. Rather than attempting to encode tail objectives entirely into a learned model, we separate *optimization* (predictive scheduling under nominal conditions) from *containment* (explicit intervention when the system enters a high-risk regime). Using a simulation framework modeling dynamic workloads and contention, we show that guarded scheduling reduces p99 latency by 47.3% relative to the predictive baseline, with a modest increase in mean latency.

In summary, this paper makes the following contributions:

- We identify a fundamental limitation of predictive scheduling: optimizing average-case objectives can underreact to rare high-impact states, enabling tail amplification under contention.
- We propose a guarded scheduling framework that introduces explicit runtime policy enforcement via mild and severe overrides triggered by queue-derived risk signals.
- We demonstrate empirically that guarded scheduling improves tail latency (p99) while maintaining competitive average-case performance.

2 System Design

We consider an online scheduling setting in which jobs arrive dynamically from multiple competing streams and contend for a shared processing resource under latency-sensitive conditions. Each job is associated with one of three stream classes: *control*, *burst*, and *background*, characterized by distinct arrival rates, service-time distributions, and base priorities. The scheduler must decide which queued job to serve next at each decision point, operating under multiple objectives: low mean response time, bounded tail latency, and state freshness. These objectives play a role analogous to service-level objectives (SLOs) commonly used to drive resource-management decisions in production systems [14].

Our design separates scheduling into two conceptual layers: a *predictive optimization layer* and a *guarded enforcement layer*. Under nominal conditions, the predictive layer exploits regularities in workload behavior to make efficient decisions. The guarded layer monitors runtime risk signals and intervenes when queue dynamics drift into an unsafe regime, providing lightweight policy enforcement to contain tail risk, mirroring a broader systems pattern of introducing explicit enforcement points to maintain predictable performance under contention [2, 9].

2.1 Observation Space

At each scheduling decision point, the system constructs a fixed-size observation vector capturing the current pipeline state. This vector consists of 13 features organized as follows:

- **Per-stream queue depths** (3 features): the number of queued jobs for each stream class.
- **Per-stream recent arrival counts** (3 features): the number of arrivals per stream within a trailing 100 ms sliding window, capturing short-term load dynamics.

- **Per-stream mean waiting times** (3 features): the mean elapsed wait of queued jobs per stream at the current decision point.
- **Per-stream maximum waiting times** (3 features): the maximum elapsed wait of queued jobs per stream, serving as a direct indicator of emerging tail risk.
- **Global freshness lag** (1 feature): the elapsed time since the most recently applied event timestamp, reflecting state staleness.
- **Last action** (1 feature): the stream class most recently selected, encoded as an integer identifier.

This observation is appended to a sliding history window of length $H = 10$, forming the sequential input to the predictive model. When fewer than H history steps are available, as occurs during system startup, the scheduler falls back to FIFO.

2.2 Predictive Scheduler

The predictive scheduler uses a learned cost model, **COSTPREDICTORLSTM**, to rank candidate actions at each decision point. The model consists of an LSTM encoder with hidden dimension 32 followed by a two-layer MLP with a ReLU activation. At inference time, the LSTM encodes the H -step observation history into a fixed-size hidden state $\mathbf{h} \in \mathbb{R}^{32}$, which is concatenated with a one-hot encoding of the candidate action $\mathbf{a} \in \{0, 1\}^3$ and passed through the MLP to produce three predicted cost components:

$$[\hat{c}_{\text{wait}}, \hat{c}_{\text{fresh}}, \hat{c}_{\text{tail}}] = \text{MLP}([\mathbf{h}; \mathbf{a}]). \quad (1)$$

Training. The model is trained offline via supervised learning on trajectories generated from the simulation environment. Training examples are collected by executing a FIFO baseline and, at each decision point, computing a rollout cost for each available action over a horizon of $K = 5$ steps using FIFO continuation. The rollout produces three scalar targets: cumulative wait cost, cumulative freshness lag, and maximum observed waiting time over the horizon. Targets are normalized to zero mean and unit variance prior to training. The training loss is a weighted mean squared error:

$$\mathcal{L} = \mathcal{L}_{\text{wait}} + \mathcal{L}_{\text{fresh}} + 5 \cdot \mathcal{L}_{\text{tail}}, \quad (2)$$

where the tail component is upweighted by a factor of 5 to emphasize rare high-cost states. The model is trained with an 80/20 train-test split, the Adam optimizer [8] with learning rate 10^{-3} , batch size 64, and 20 epochs. Across five seeds, the number of generated training examples ranged from 92,303 to 93,900, reflecting variation in job arrival counts across runs. Training loss converged variably across seeds, with some seeds exhibiting stable monotonic decrease and others showing oscillation in later epochs, consistent with the difficulty of predicting tail costs under stochastic arrivals. Despite this variability, the guarded scheduler’s consistent performance across seeds suggests that the guard layer provides robustness to prediction quality variation. The model from the final training epoch is used for inference in all experiments.

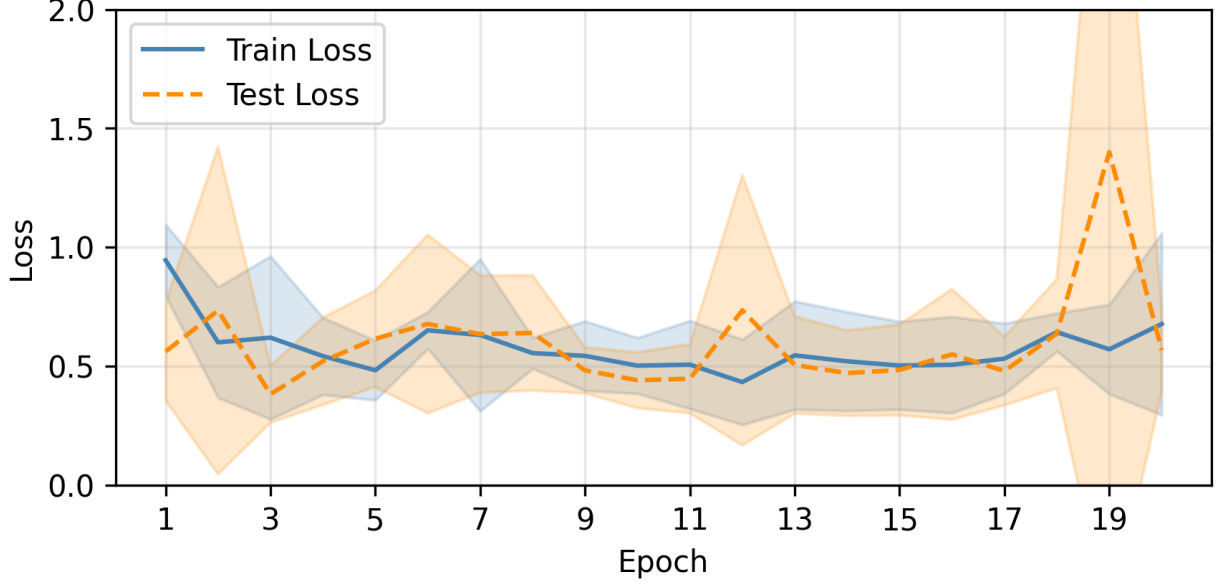


Figure 1: Training and test loss across 20 epochs, averaged over five seeds. Shaded regions indicate ± 1 standard deviation. Loss convergence varies across seeds, consistent with the stochastic nature of tail cost prediction.

Inference. At inference time, the scheduler scores each available action a as:

$$\text{score}(a) = w_{\text{wait}} \hat{c}_{\text{wait}}(a) + w_{\text{fresh}} \hat{c}_{\text{fresh}}(a) + w_{\text{tail}} \hat{c}_{\text{tail}}(a) - \lambda w_{\text{max}}(a), \quad (3)$$

where $w_{\text{wait}} = 1.0$, $w_{\text{fresh}} = 0.2$, $w_{\text{tail}} = 8.0$, and $\lambda = 0.08$ is a wait-bias coefficient. The tail weight at inference ($w_{\text{tail}} = 8.0$) is tuned independently from the training loss upweighting ($5\times$), as the two serve distinct roles: the loss upweighting shapes learned representations during training, while the inference weight governs action selection at runtime. The term $w_{\text{max}}(a)$ denotes the maximum observed waiting time among queued jobs belonging to stream a ; subtracting it discounts streams already experiencing elevated wait, reducing systematic starvation. The action with the lowest score is selected, and among jobs of the chosen stream, the earliest-arriving job is served first.

2.3 Lookahead Scheduler

The lookahead scheduler extends the predictive scheduler with a model-predictive control style rollout. Rather than selecting the action with the lowest single-step predicted cost, it evaluates sequences of actions over a horizon of depth $d = 3$ with branching factor $b = 2$. At each rollout step, the top- b candidate actions are identified using the scoring function in Equation 3, and the scheduler recursively evaluates their downstream cost. The per-step rollout cost is:

$$c_{\text{step}} = \left(\frac{\hat{c}_{\text{tail}}}{100} \right)^3, \quad (4)$$

a strongly convex function that heavily penalizes large predicted tail values. The root action with the lowest cumulative rollout cost is selected. This baseline tests whether multi-step lookahead using the same underlying model improves tail control beyond single-step prediction.

2.4 Guarded Scheduler

The guarded scheduler augments the predictive scheduler with threshold-based runtime intervention. The guard layer monitors $w_{\max}(t) = \max_i w_i(t)$, the maximum oldest waiting time across all streams at time t , and selectively overrides predictive decisions when the system enters a high-risk regime. Two intervention levels are defined by thresholds $\tau_m < \tau_s$:

Mild guard. When $w_{\max}(t) \geq \tau_m$ or the total queue length exceeds a depth threshold $Q = 25$ jobs, the scheduler identifies the most at-risk stream $s^* = \arg \max_i w_i(t)$. If the predictive scheduler has already selected a job from s^* , the decision is preserved. Otherwise, the guard compares the oldest wait of the predictive choice against s^* : if the disparity exceeds a safety margin of 2000 ms, the guard overrides the predictive decision in favor of s^* .

Severe guard. When $w_{\max}(t) \geq \tau_s$, the scheduler unconditionally selects the earliest-arriving job from s^* , prioritizing containment over efficiency.

Outside guard conditions, the scheduler follows the predictive decision. This policy is intentionally asymmetric: prediction governs the fast path, while guarded overrides are reserved for exceptional states, consistent with systems that introduce lightweight enforcement points to bound harmful queue evolution [2, 9].

2.5 Design Rationale

The central premise of our design is that average-case optimization and tail-risk mitigation should not be treated as the same problem. The predictive layer is effective when system behavior aligns with learned patterns, but it does not inherently prevent worst-case outcomes. The guard layer provides a corrective mechanism that ensures no stream experiences unbounded delay, activating only when risk signals indicate that prediction-only control is insufficient [3]. Because enforcement and observability can introduce overhead if deployed indiscriminately, our guard triggers are simple, state-driven, and inexpensive to evaluate online [15].

3 Methodology

We evaluate our scheduling approach using a discrete-event simulation framework designed to model dynamic workloads and shared-resource contention. The simulator generates streams of jobs with varying arrival patterns and processing requirements, allowing us to analyze scheduler behavior under both typical and adverse conditions. Our evaluation emphasizes tail behavior, since rare contention-induced events can dominate end-to-end responsiveness in large systems [3].

3.1 Simulation Setup

The simulator models a single shared processing resource serving three concurrent job streams: *control*, *burst*, and *background*. Each stream is characterized by distinct arrival dynamics, service-time distributions, and base priorities, summarized in Table 1.

Interarrival times are sampled from an exponential distribution with the mean specified above, producing Poisson arrival processes for each stream under normal conditions. Service times are drawn uniformly from the intervals shown. The burst stream introduces a sustained high-load phase from $t = 1500$ ms to $t = 3500$ ms, during which its mean interarrival time drops to 0.8 ms, producing a sharp contention spike that stresses the scheduler’s ability to contain tail

Table 1: Workload configuration for each job stream.

Stream	Base Priority	Mean Interarrival (ms)	Service Time (ms)
Control	10.0	8.0	$\mathcal{U}(2, 4)$
Burst	5.0	18.0	$\mathcal{U}(6, 10)$
Background	1.0	12.0	$\mathcal{U}(3, 6)$

amplification. This burst phase is the primary source of extreme latency events in our evaluation and is designed to expose the failure modes of purely predictive scheduling under distribution shift.

Jobs arrive over time and are placed into per-stream queues. At each scheduling decision point, the scheduler selects a job from the set of queued candidates according to the policy being evaluated. The simulation runs for a total duration of 50,000 ms and is repeated across five independent random seeds; reported results reflect aggregated statistics over these runs. All experiments were conducted on a commodity laptop running Ubuntu 22.04 with 16GB RAM. Each seed required approximately 90 minutes of wall-clock time to complete.

We compare three scheduling policies:

- **Predictive:** The LSTM-based scheduler described in Section 2.2, which selects actions by minimizing a weighted combination of predicted wait, freshness, and tail cost (Equation 3).
- **Lookahead:** A model-predictive control style extension of the predictive scheduler that evaluates action sequences over a depth-3 horizon with branching factor 2, selecting the root action with lowest cumulative predicted tail cost (Section 2.3).
- **Guarded:** Our proposed approach, which augments the predictive scheduler with threshold-based runtime enforcement via mild and severe override mechanisms triggered by queue-derived risk signals (Section 2.4).

3.2 Metrics

We evaluate each scheduling policy using the following metrics:

- **Mean latency:** Average job response time.
- **Tail latency:** High-percentile latency (p95 and p99), capturing worst-case behavior.
- **Freshness:** Delay between job arrival and service relative to stream dynamics.
- **Variance:** Stability of response times across the workload.

Tail latency is the primary metric of interest, as it reflects the system’s ability to prevent rare but extreme delays under contention [3].

3.3 Guard Activation Analysis

In addition to latency metrics, we track the frequency of guard activations during execution. Specifically, we record the number of mild and severe overrides triggered by the guarded scheduler. These measurements provide insight into when and how often the system enters high-risk states requiring explicit intervention.

Rather than treating overrides as anomalies, we interpret them as indicators that the system has entered regimes where prediction-only control is insufficient and explicit policy enforcement becomes necessary [2]. Because measurement and control logic can introduce non-trivial overhead in real deployments, we focus on simple, state-driven triggers that are inexpensive to evaluate online [15].

3.4 Experimental Procedure

Each configuration is evaluated across multiple runs with different random seeds to account for stochastic variability in workload generation. Reported results include aggregated statistics over these runs, ensuring that observed improvements are not artifacts of a particular execution trace. We analyze both average-case behavior and worst-case outcomes, with particular emphasis on identifying scenarios where predictive scheduling fails and guarded intervention alters system dynamics.

3.5 Guard Configuration and Parameters

We evaluate four configurations (A–D) that vary guard thresholds to explore the tradeoff between average performance and tail latency reduction. The configurations are summarized in Table 2. This parameter sweep is motivated by the practical reality that enforcing predictable tail behavior often requires carefully calibrated intervention policies [12].

Table 2: Guard threshold configurations evaluated in this work. Thresholds are defined in milliseconds of observed waiting time.

Config	Mild Threshold (ms)	Severe Threshold (ms)	Description
A	10000	14000	Balanced intervention (baseline)
B	9000	14000	Earlier mild intervention
C	10000	13000	Earlier severe intervention
D	9000	13000	Aggressive intervention

These configurations span a range of intervention strategies, from conservative (Config A) to more aggressive policies (Config D). This allows us to examine how the timing and frequency of guard activation affect both mean performance and tail behavior.

4 Analytical Model of Guarded Scheduling

To better understand the behavior of guarded scheduling, we introduce a simplified analytical model of the system. The goal of this model is not to capture all system dynamics, but to formalize the conditions under which predictive scheduling fails and guarded intervention becomes necessary. This separation mirrors a broader systems theme: predictive or heuristic optimization can be effective in nominal regimes, but enforcing predictable behavior under contention often requires explicit runtime policy enforcement [2, 9].

4.1 System Model

We consider a system with N independent job streams. Each stream i maintains a queue of pending jobs, and we define the state of the system at time t by the set of queue wait times:

$$w_i(t) = \text{waiting time of the oldest job in stream } i.$$

At each decision point, the scheduler selects a stream j from which to serve the next job.

4.2 Predictive Scheduling

Let $C(j, t)$ denote the predicted cost of selecting stream j at time t . A predictive scheduler selects:

$$j^*(t) = \arg \min_j C(j, t).$$

This formulation captures a broad class of learned or model-driven schedulers that aim to optimize expected (average-case) performance using historical signals [10]. However, because $C(j, t)$ is derived from historical or learned signals, it may not accurately reflect emerging imbalance in queue states. In particular, streams with large $w_i(t)$ may be de-prioritized if their predicted cost does not sufficiently penalize delay.

4.3 Tail Risk and Amplification

We define a high-risk state as one in which at least one stream exhibits excessive waiting time:

$$\max_i w_i(t) > \tau,$$

for some threshold τ . Under purely predictive scheduling, such states can persist or worsen if the scheduler repeatedly selects streams with lower predicted cost. This leads to tail amplification, where waiting times grow disproportionately for certain streams, and rare events dominate high-percentile latency [3].

4.4 Guarded Scheduling Policy

The guarded scheduler augments predictive selection with threshold-based intervention. We define two thresholds:

$$\tau_m < \tau_s,$$

corresponding to mild and severe risk levels. At each decision point, the scheduler proceeds as follows:

1. Compute the predictive choice $j^*(t)$.
2. Compute $w_{\max}(t) = \max_i w_i(t)$.

Mild guard: If $w_{\max}(t) > \tau_m$, the scheduler evaluates whether the predictive choice sufficiently serves the most delayed stream. If not, it may override the predictive choice in favor of the stream with largest $w_i(t)$.

Severe guard: If $w_{\max}(t) > \tau_s$, the scheduler selects:

$$\hat{j}(t) = \arg \max_i w_i(t),$$

prioritizing the most delayed stream to prevent further escalation. Otherwise, the scheduler follows the predictive decision:

$$\hat{j}(t) = j^*(t).$$

This policy can be interpreted as a lightweight enforcement mechanism: prediction governs the fast path, while the guard introduces an explicit containment action when risk becomes large. Similar enforcement points are common in systems that centralize policy control to maintain predictable performance under contention [2, 9].

4.5 Interpretation

This policy introduces an explicit separation between two objectives:

- **Optimization:** The predictive model minimizes expected cost under typical conditions [10].
- **Containment:** The guard mechanism bounds extreme queue growth during high-risk states.

The predictive layer is effective when system behavior aligns with learned patterns, but it does not inherently prevent worst-case outcomes. The guard layer provides a corrective mechanism that ensures no stream experiences unbounded delay, and is calibrated to act only when risk signals indicate that prediction-only control is insufficient.

4.6 Implications for Tail Latency

The key effect of guarded scheduling is to limit the duration of high-risk states. By prioritizing streams with large $w_i(t)$ once thresholds are exceeded, the scheduler prevents sustained accumulation of delay. While guarded intervention may temporarily deviate from optimal average-case decisions, it reduces the likelihood of extreme latency outcomes, resulting in improved tail behavior without requiring the predictive model to explicitly encode worst-case constraints [3].

More broadly, this suggests that tail control in dynamic systems benefits from a hybrid design: prediction for efficiency in nominal regimes, paired with explicit enforcement mechanisms for robustness under contention [2, 9].

5 Results

We evaluate the guarded scheduler under Config A, a balanced intervention strategy with mild threshold 10,000 ms and severe threshold 14,000 ms. These thresholds were selected based on observed queue dynamics during pilot runs, where maximum waiting times consistently exceeded 10,000 ms during the burst phase and severe congestion was indicated by waiting times above 14,000 ms, and represent a representative starting point rather than a configuration optimized to this specific workload. Because rare tail events can dominate perceived responsiveness in large systems, we emphasize high-percentile latency (p95/p99) throughout [3].

5.1 Baseline Comparison

We first compare the predictive and lookahead schedulers. As shown in Fig. 2, both policies exhibit similar mean latency, with lookahead achieving slightly lower mean latency (approximately 15,698 ms vs. 17,184 ms). Notably, at extended evaluation duration, lookahead provides meaningful tail improvement over predictive, with p99 latency of approximately 46,638 ms versus 53,451 ms, suggesting that multi-step rollout provides some tail benefit under sustained contention that is not visible over shorter horizons. Nevertheless, both approaches yield p99 latency exceeding 46 seconds, indicating that predictive estimation and limited lookahead alone are insufficient to control extreme delays in the presence of contention and stochasticity [3, 10].

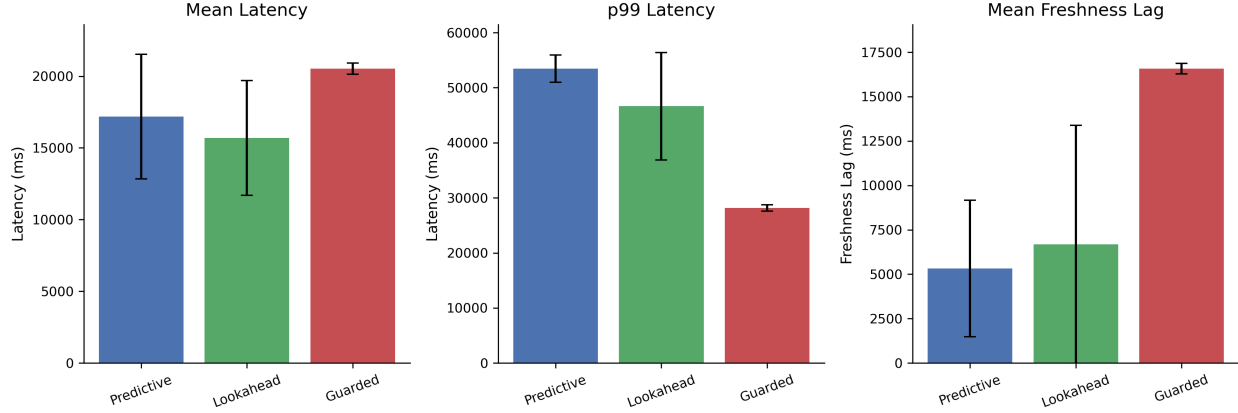


Figure 2: Comparison of mean latency, p99 latency, and mean freshness lag across the scheduling policies. Predictive and lookahead policies achieve lower mean latency, while guarded scheduling reduces tail latency at the cost of higher average delay and freshness lag.

5.2 Guarded Scheduling Performance

We next evaluate the guarded scheduler under Config A, which represents a balanced intervention strategy. As shown in Fig. 2 and Fig. 3, guarded scheduling reduces p99 latency from approximately 53,451 ms (predictive) to 28,157 ms, a reduction of 47.3%, while also substantially improving p95 latency from 50,075 ms to 27,718 ms. Beyond percentile improvements, guarded scheduling reduces latency standard deviation from 16,871 ms to 7,478 ms, indicating markedly more consistent behavior across jobs. This improvement comes at the cost of increased mean latency, which rises from approximately 17,184 ms to 20,522 ms. These results demonstrate a clear but favorable tradeoff: guarded scheduling sacrifices modest average performance to achieve substantial reductions in extreme delays and variance [3].

Notably, the guarded scheduler demonstrates remarkable consistency across seeds. Mean latency varies by only ± 391 ms across five independent runs, and p99 latency varies by only ± 577 ms, despite substantial seed-to-seed variation in workload arrival patterns and training trajectories. This consistency stands in sharp contrast to the lookahead scheduler, whose p99 latency varies by $\pm 9,737$ ms across seeds — a range more than sixteen times larger. This instability likely arises because multi-step rollout compounds single-step prediction errors across the horizon, amplifying seed-to-seed variation in model quality into large swings in tail behavior. The predictive scheduler similarly exhibits high variance, with p99 std of $\pm 2,472$ ms. The stability of guarded scheduling across seeds suggests that the guard layer’s threshold-based enforcement provides a reliable performance floor that is robust to both workload variation and prediction quality, rather than depending on favorable random seeds or well-trained models to achieve tail control.

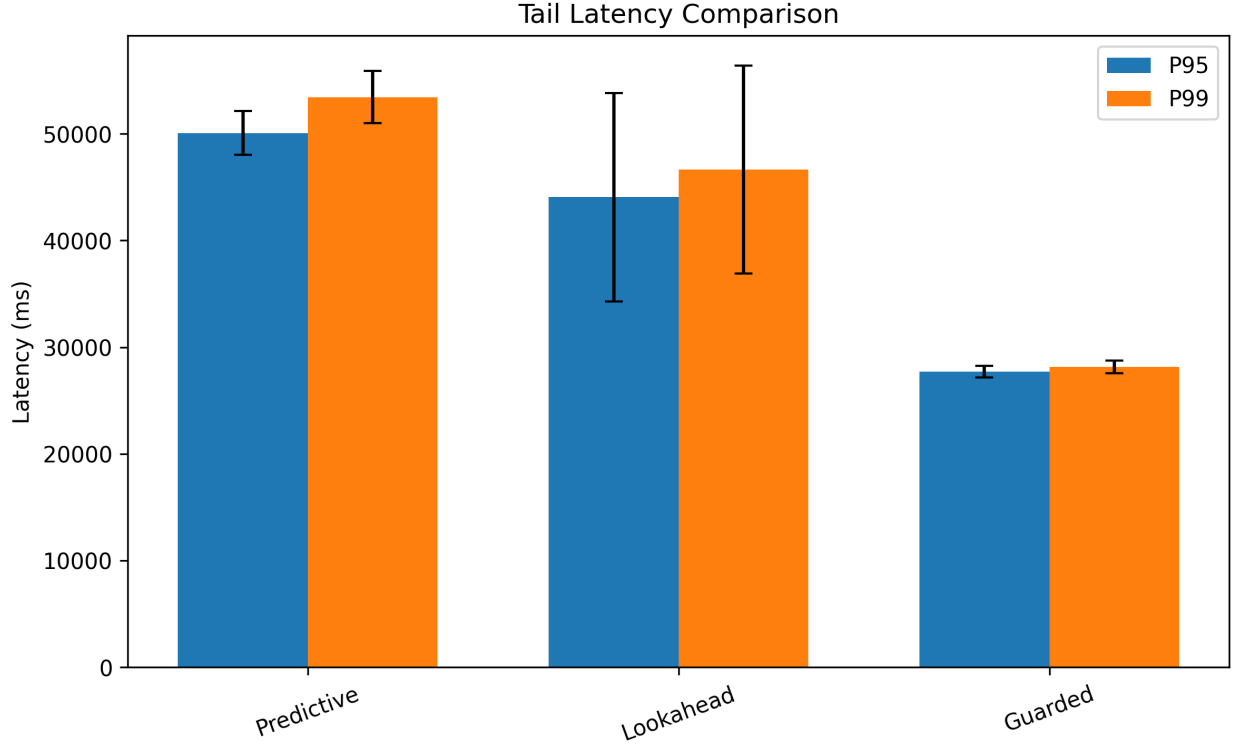


Figure 3: Tail-latency comparison across scheduling policies. Guarded scheduling improves both p95 and p99 latency relative to predictive and lookahead baselines, showing that explicit runtime intervention is effective for reducing extreme delays.

The underlying mechanism is illustrated in Fig. 4, which shows that the guard actively intervenes during execution. Under Config A, the guarded scheduler triggered a mean of 12,901 severe overrides and 1,234 mild overrides per run, with mild decisions preserved by the predictive scheduler in 221 cases on average. This confirms that performance gains arise from explicit runtime intervention rather than passive scheduling behavior, consistent with systems that introduce enforcement points to improve predictability under uncertainty [2, 9].

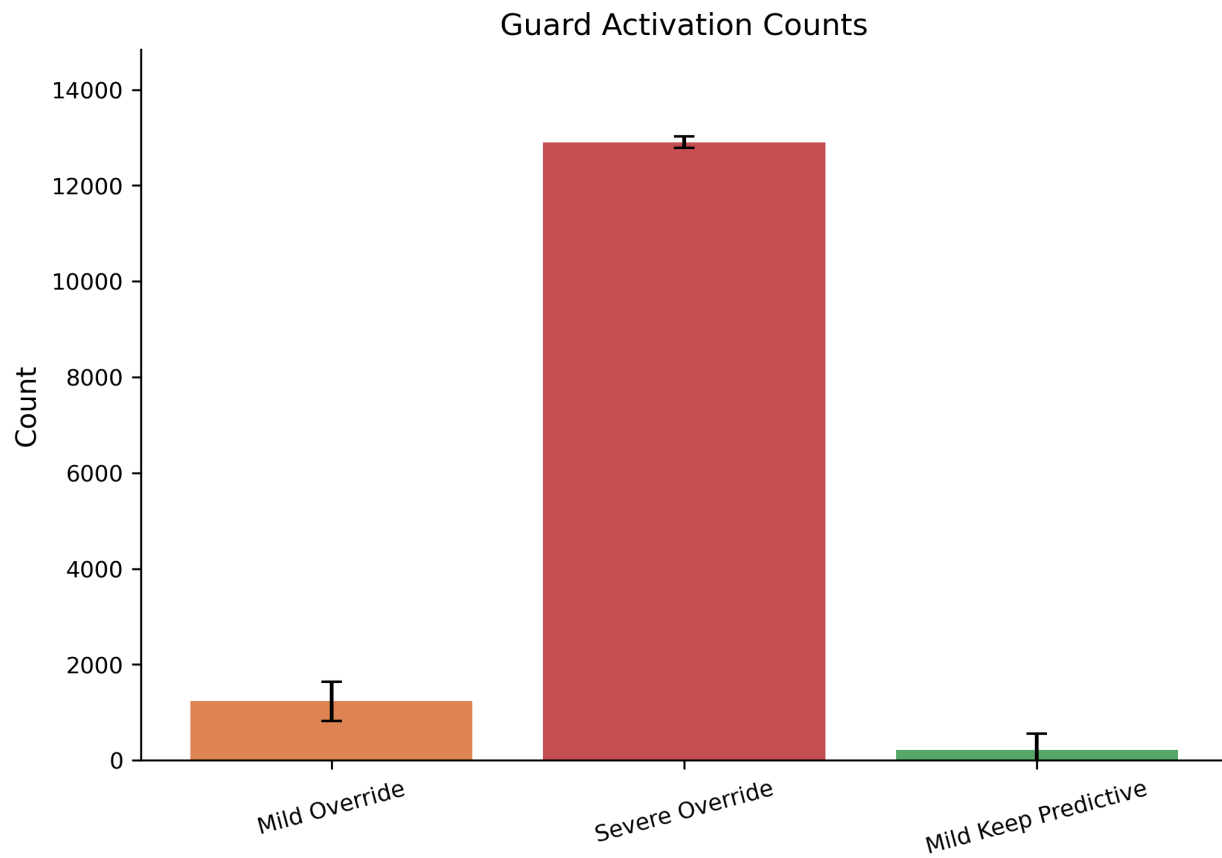


Figure 4: Guard activation behavior under the guarded scheduler. Mild and severe overrides occur frequently enough to materially affect tail behavior, while retained predictive decisions show that the scheduler does not fully abandon predictive control.

5.3 Sensitivity to Guard Configuration

To assess robustness, we evaluate additional configurations (B–D) that vary the timing of mild and severe intervention thresholds. Results are summarized in Fig. 2 and Fig. 5.

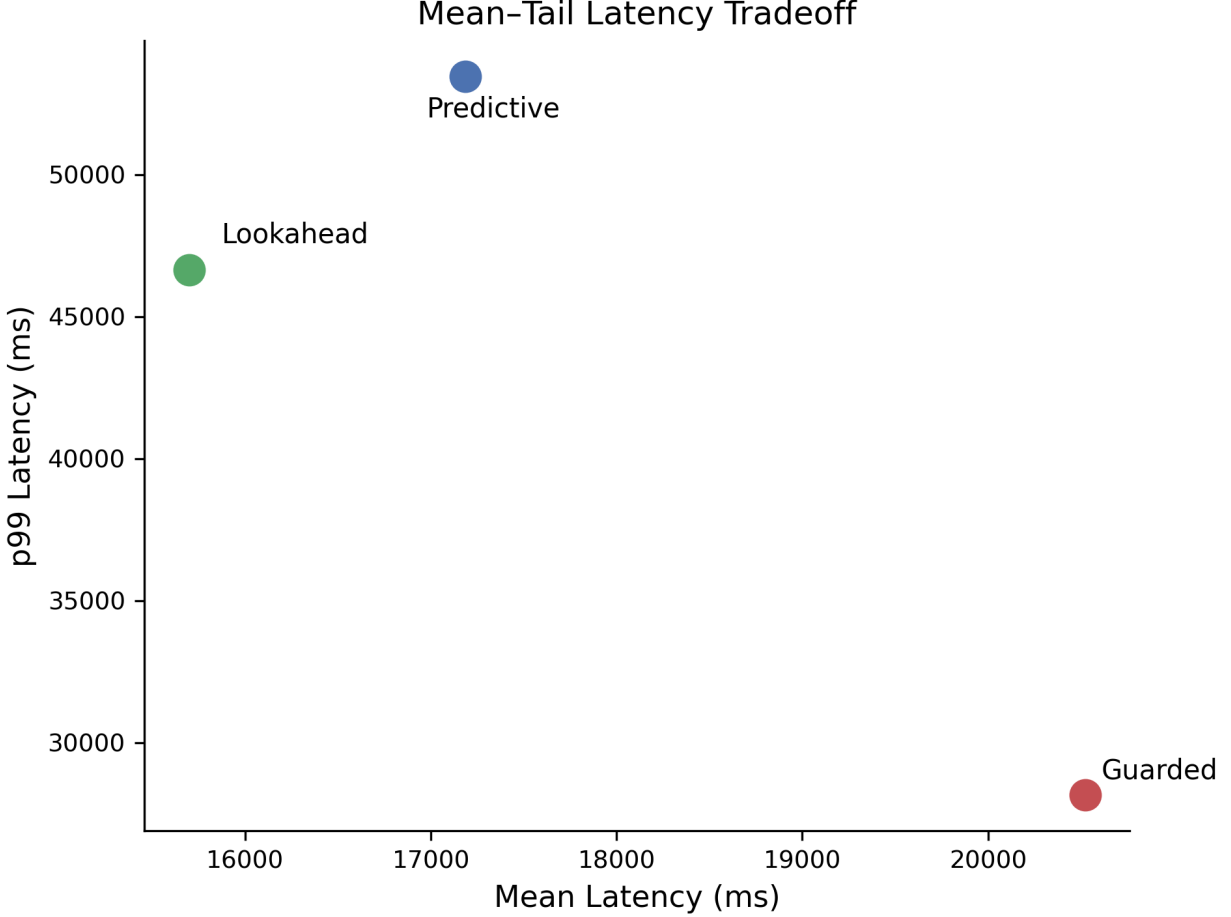


Figure 5: Tradeoff between average performance and tail behavior. Guarded scheduling shifts the operating point toward lower p99 latency at the cost of increased mean latency, illustrating the central tradeoff explored in this paper.

We observe that increasing intervention aggressiveness shifts scheduler behavior without substantially improving tail latency beyond Config A. In particular, Config B introduces more frequent mild overrides but yields negligible improvement in p99 latency, while Configs C and D increase severe intervention frequency and achieve only marginal additional reductions. These findings suggest diminishing returns from aggressive guard policies. Moderate configurations (e.g., Config A) capture most of the achievable tail improvement while maintaining lower intervention overhead.

5.4 Distributional Analysis

To further characterize scheduler behavior, Fig. 6 presents the cumulative distribution function (CDF) of job latency. The guarded scheduler shifts the upper tail of the distribution leftward, reducing the frequency of extreme delays while slightly degrading lower-percentile performance. This confirms that guarded scheduling primarily targets high-latency events rather than uniformly improving all jobs, which is desirable when tail behavior dominates user experience or system reliability [3].

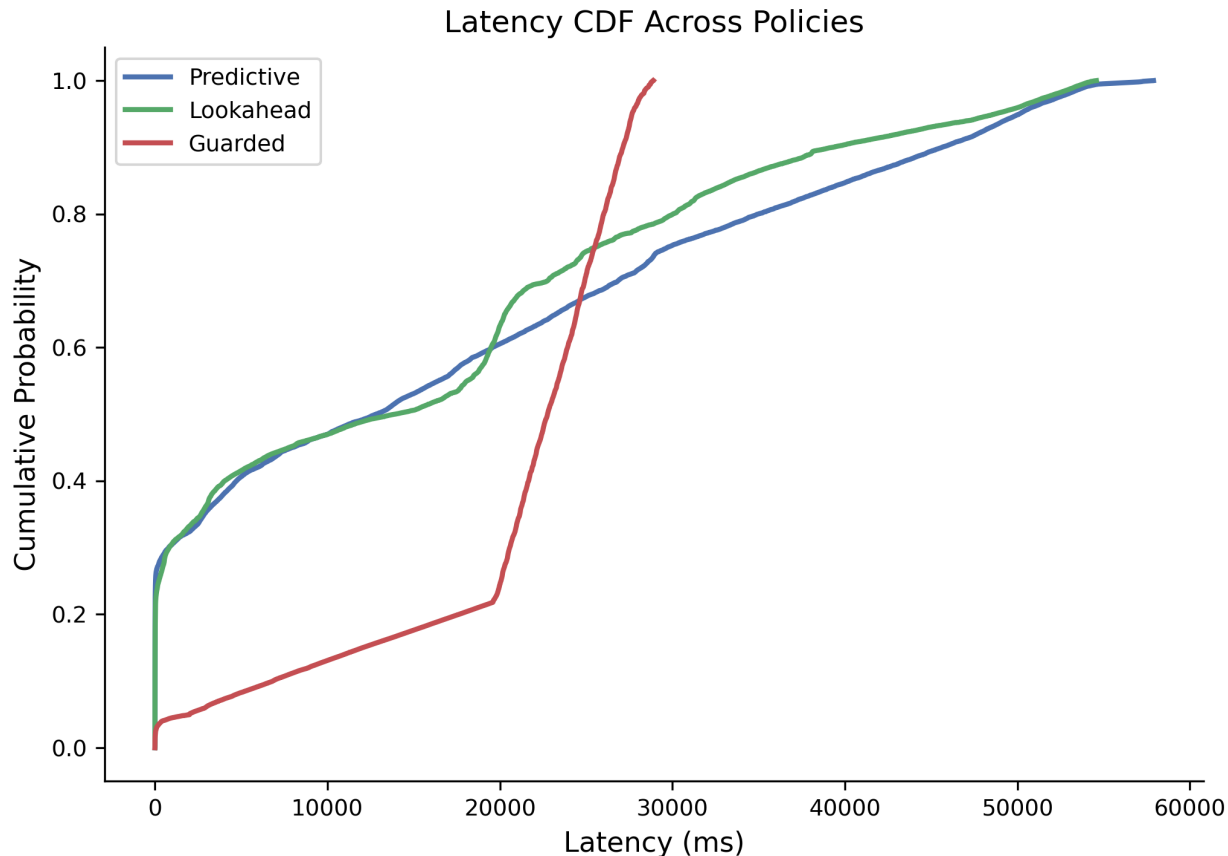


Figure 6: Cumulative distribution function (CDF) of job latency across scheduling policies. Guarded scheduling shifts the upper tail of the distribution leftward, reducing the frequency of extreme delays while slightly degrading lower-percentile performance.

5.5 Summary of Tradeoffs

Overall, guarded scheduling introduces a controllable tradeoff between mean latency and tail behavior. Moderate guard configurations significantly reduce extreme delays with limited overhead, while more aggressive settings provide only marginal additional benefit. This highlights the importance of calibrated intervention policies in managing tail latency under contention: predictive scheduling provides efficiency under nominal conditions, while explicit runtime enforcement improves robustness when the system enters high-risk regimes [10, 2, 9].

6 Discussion

Our results demonstrate that predictive scheduling alone is insufficient to control tail latency in dynamic environments. While predictive and lookahead schedulers achieve strong average performance, they fail to prevent rare but significant delays, as evidenced by consistently high p99 latency. This outcome reflects a well-known property of large systems: rare events can dominate end-to-end latency, and optimizing for the mean does not guarantee tail control [3]. This substantial p99 reduction occurs under a deliberately adversarial burst phase designed to stress scheduler resilience under distribution shift. In latency-sensitive systems where tail

events compound across pipeline stages, even small reductions in worst-case latency carry disproportionate practical impact [3].

The mean latency increase from 17,184 ms to 20,522 ms represents a 19.4% cost in average-case performance. In practice, this tradeoff is favorable for latency-sensitive systems where tail events carry outsized impact on user experience and service-level objectives. A system operator facing a choice between a scheduler that achieves low mean latency but allows p99 to exceed 53 seconds, versus one that accepts a modest mean increase while bounding p99 below 29 seconds, would in most production contexts prefer the latter. This is consistent with how SLO-driven systems are evaluated in practice: violations at high percentiles are weighted more heavily than average-case degradation, since a single tail event can affect many downstream components in a pipelined system [3, 14]. Furthermore, the reduction in latency standard deviation from 16,871 ms to 7,478 ms indicates that guarded scheduling produces more predictable behavior overall, which simplifies capacity planning and reduces the likelihood of cascading delays under sustained load. The guarded scheduler addresses this limitation by introducing explicit runtime intervention. Rather than relying solely on prediction accuracy, the system enforces corrective behavior when waiting times exceed predefined thresholds. This shifts the scheduling paradigm from purely predictive optimization to hybrid predictive-reactive control, where prediction governs the fast path and enforcement governs exceptional regimes. This separation parallels a broader trend in datacenter systems toward centralized visibility and explicit policy enforcement to maintain predictability under contention [2, 9].

A key insight from our evaluation is that moderate intervention provides the most effective tradeoff. Config A achieves substantial reductions in tail latency while maintaining acceptable mean performance. Increasing intervention aggressiveness yields diminishing returns, suggesting that once extreme delays are sufficiently suppressed, additional overrides primarily introduce overhead without meaningful benefit. In practice, this emphasizes that intervention policies should be calibrated and iterated carefully, similar to how systems work has highlighted the importance of rapid feedback loops and structured tuning when evolving scheduling behavior [12].

The distributional analysis further supports this interpretation. Improvements are concentrated in the upper tail of the latency distribution, indicating that guarded scheduling selectively targets pathological cases rather than uniformly improving all jobs. This behavior is desirable in systems where worst-case performance dominates user experience or system reliability [3].

Overall, these findings suggest that effective scheduling in uncertain environments benefits from combining prediction and enforcement. Predictive approaches optimize average behavior using historical signals [10], while guard mechanisms provide robustness against tail risk by explicitly containing high-risk states [2, 9]. Finally, since enforcement and observability can introduce overhead if deployed indiscriminately, lightweight triggers and careful placement of control logic remain essential [15].

7 Related Work

Tail Latency in Distributed Systems. The challenge of tail latency in large-scale systems has been extensively studied since Dean and Barroso identified it as a first-class systems concern [3]. Their analysis establishes that rare events dominate end-to-end responsiveness in systems with many parallel components, motivating designs that explicitly target high-percentile latency rather than mean performance. Subsequent work has addressed tail latency through redundancy [3], hedged requests, and load balancing, but these approaches primarily reduce

the impact of slow servers rather than addressing queue-level scheduling as a source of tail amplification. More recent work targets tail latency through predictive component-level scheduling [13] and system-wide scheduling frameworks [6], but these systems focus on service-level routing decisions rather than fine-grained job-level arbitration under contention. Our work is distinguished by its focus on queue-derived risk signals as a direct mechanism for tail control within a single scheduling loop, without relying on redundancy or replica-level decisions.

Learning-Based Scheduling. A growing body of work applies machine learning to scheduling problems in distributed systems. Mao et al. [10] demonstrate that deep reinforcement learning can effectively guide resource allocation decisions, improving average-case performance by exploiting historical system signals. Decima [11] extends this direction to DAG-structured cluster workloads, using graph neural networks and RL to learn workload-specific policies that outperform hand-tuned heuristics on average job completion time. These systems demonstrate the power of learning-based approaches under typical workload conditions, but they optimize primarily for mean objectives and do not explicitly address tail behavior under distribution shift or bursty contention. Predictive component-level scheduling [13] uses analytical performance models to forecast component latency and reallocate tasks, reducing tail latency in parallel online services, but relies on explicit performance models rather than learned predictors and does not incorporate runtime enforcement. Our work builds on the learning-based scheduling paradigm but identifies a fundamental limitation: optimizing average-case objectives can systematically underreact to rare high-impact states, and addresses this through explicit runtime enforcement rather than improved prediction alone.

Real-Time Scheduling and Budget Enforcement. The real-time systems community has developed principled frameworks for constructing schedulers with formal worst-case guarantees. Classical results establish schedulability conditions under fixed-priority and earliest-deadline-first policies, providing provable bounds on response times for periodic and sporadic task sets [1]. More recent work addresses mixed workloads combining hard and soft real-time tasks, using server budgets and hierarchical scheduling to isolate timing-critical components while preserving efficiency for best-effort traffic [7]. Anderson and colleagues have pursued principled budget enforcement as a mechanism for constructing schedulers with explicit worst-case guarantees [5]. These approaches provide strong formal guarantees but assume workloads with known timing parameters and periodic structure. Our setting involves dynamic, data-driven workloads with heterogeneous and unpredictable arrival patterns, where learned predictors replace explicit timing models. Our guarded enforcement mechanism draws conceptual inspiration from budget-based intervention in real-time systems, adapting the idea of threshold-triggered containment to a learning-augmented scheduling context.

Hybrid Predictive-Reactive Control. Several systems have explored hybrid architectures that combine prediction with reactive enforcement. mRPC [2] relocates policy enforcement into a managed system service, demonstrating that separating optimization from enforcement can improve predictability under contention without imposing unnecessary overhead. Harmonic [9] uses hardware-assisted isolation to enforce performance boundaries in multi-tenant RDMA environments, illustrating the value of explicit enforcement points for tail-resilient behavior. Enoki [12] provides a framework for rapid scheduler development in Linux, enabling iterative refinement of scheduling policies through structured feedback loops. The model-predictive control (MPC) paradigm offers a formal framework for combining prediction with constraint

enforcement, and has been applied to resource scheduling problems where prediction and constraint enforcement must be jointly satisfied [4]. Our guarded scheduling framework shares the hybrid structure of these systems: prediction governs the fast path under nominal conditions, while explicit enforcement intervenes when risk signals indicate that prediction-only control is insufficient. Our contribution is the application of this pattern to fine-grained job-level scheduling under stochastic multi-stream contention, with runtime risk signals derived directly from observable queue state.

7.1 Limitations

This work evaluates guarded scheduling within a discrete-event simulation environment. While the simulator captures key dynamics of multi-stream contention, bursty arrivals, and stochastic service times, it does not model all sources of variability present in real deployments, including network-induced jitter, hardware contention, operating system scheduling interference, and non-stationary workload drift over long time horizons. Evaluation on a physical testbed or using production workload traces would strengthen the empirical claims, and we identify this as the primary direction for future work. Additionally, FIFO is excluded from the policy comparison because it serves as the data-generating policy for supervised training; including it as an evaluation baseline would conflate the training procedure with the policy comparison, obscuring the contribution of the learned cost model.

The predictive model is trained and evaluated on trajectories generated by the same simulator, which limits the ability to assess generalization under significant distributional shift. In practice, a model deployed on live traffic may encounter workload regimes not represented in its training distribution, potentially increasing guard activation frequency and overhead. Online adaptation or periodic retraining may be necessary to maintain predictive accuracy in non-stationary environments.

Guard thresholds τ_m , τ_s , and Q are configured statically and evaluated through a parameter sweep. While our sensitivity analysis (Section 5.3) demonstrates that moderate configurations capture most of the achievable tail improvement, optimal threshold selection in practice may require workload-specific tuning. Adaptive thresholding driven by online performance feedback is a natural extension and is left for future work.

Finally, our evaluation focuses on latency-based objectives and does not explicitly assess throughput, fairness, or resource utilization tradeoffs. In settings where multiple objectives must be jointly satisfied under explicit service-level constraints, extending the guarded framework to multi-objective optimization remains an open problem [14].

8 Conclusion

We present a guarded scheduling framework that augments predictive decision-making with threshold-based runtime intervention. Our results show that this approach reduces tail latency while maintaining competitive average performance, addressing a central challenge in distributed systems where rare events disproportionately impact responsiveness [3].

By explicitly trading mean latency for improved worst-case behavior, guarded scheduling provides a practical mechanism for managing tail risk under contention. We further show that moderate intervention strategies capture most of the achievable benefit, while more aggressive policies yield diminishing returns. This reinforces a broader systems design principle: combine predictive optimization with explicit enforcement to maintain predictable behavior under uncertainty [10, 2, 9].

Future work will explore adaptive thresholding, integration with online learning, and extension to multi-resource scheduling environments.

References

- [1] Giorgio C. Buttazzo. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. Springer, 3rd edition, 2011.
- [2] Jingrong Chen, Yongji Wu, Shihan Lin, Yechen Xu, Xinhao Kong, Thomas Anderson, Matthew Lentz, Xiaowei Yang, and Danyang Zhuo. Remote Procedure Call as a Managed System Service. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 141–159, 2023.
- [3] Jeffrey Dean and Luiz André Barroso. The Tail at Scale. *Communications of the ACM*, 56(2):74–80, February 2013.
- [4] Carlos E. Garcia, David M. Prett, and Manfred Morari. Model Predictive Control: Theory and Practice — A Survey. *Automatica*, 25(3):335–348, 1989.
- [5] Joseph Goh and James H. Anderson. Towards Principled Budget Enforcement in Real-Time Systems. In *2024 IEEE Real-Time Systems Symposium (RTSS)*, pages 256–266. IEEE Computer Society, 2024.
- [6] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. TailClipper: Reducing tail response time of distributed services through system-wide scheduling. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*. ACM, 2024.
- [7] Christopher J. Kenna, Jonathan L. Herman, Björn B. Brandenburg, Alex F. Mills, and James H. Anderson. Soft Real-Time on Multiprocessors: Are Analysis-Based Schedulers Really Worth It? In *32nd IEEE Real-Time Systems Symposium (RTSS 2011)*. IEEE Computer Society, 2011.
- [8] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*, 2015.
- [9] Jiaqi Lou, Xinhao Kong, Jinghan Huang, Wei Bai, Nam Sung Kim, and Danyang Zhuo. Harmonic: Hardware-assisted RDMA Performance Isolation for Public Clouds. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1479–1496, 2024.
- [10] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource Management with Deep Reinforcement Learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets-XV)*, pages 50–56, 2016.
- [11] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrisnan, Zili Meng, and Mohammad Alizadeh. Learning Scheduling Algorithms for Data Processing Clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19)*, pages 270–288, 2019.
- [12] Samantha Miller, Anirudh Kumar, Tanay Vakharia, Ang Chen, Danyang Zhuo, and Thomas Anderson. Enoki: High Velocity Linux Kernel Scheduler Development. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*, pages 962–980, 2024.

- [13] Cheng Wang, Bingsheng He, Huiqi Hu, and Wei Wang. PCS: Predictive Component-level Scheduling for Reducing Tail Latency in Cloud Online Services. In *2015 IEEE 35th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2015.
- [14] Yongji Wu, Matthew Lentz, Danyang Zhuo, and Yao Lu. Serving and Optimizing Machine Learning Workflows on Heterogeneous Infrastructures. *Proceedings of the VLDB Endowment (PVLDB)*, 16(3):406–419, 2022.
- [15] Xiangfeng Zhu, Guozhen She, Bowen Xue, Yu Zhang, Yongsu Zhang, Xuan Kelvin Zou, Xiongchun Duan, Peng He, Arvind Krishnamurthy, Matthew Lentz, Danyang Zhuo, and Ratul Mahajan. Dissecting Overheads of Service Mesh Sidecars. In *Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23)*, pages 142–157, 2023.